

1. $\{\omega | \omega \in \{a,b,c\}^* \text{ and the numbers of } a\text{'s and } b\text{'s and } c\text{'s occurred in } \omega \text{ are even}\}$.

根据 abc 个数的奇偶性，这里 0 代表偶数，1 代表奇数,*表示终态

	a	b	c
S*	0	0	0
A	0	0	1
B	0	1	0
C	0	1	1
D	1	0	0
E	1	0	1
F	1	1	0
G	1	1	1

$S \rightarrow aD | bB | cA | \epsilon$

$A \rightarrow aE | bC | cS$

$B \rightarrow aF | bS | cC$

$C \rightarrow aG | bA | cB$

$D \rightarrow aS | bF | cE$

$E \rightarrow aA | bG | cD$

$F \rightarrow aB | bD | cG$

$G \rightarrow aC | bE | cF$

错误人数较少

错误原因在于不知道画电路图求解

2. $\{a^i b^j | i \geq (2j+1) \text{ and } j \geq 0\}$

$b) \{a^i b^j | i \geq (2j+1), j \geq 0\}$

$a^i b^j = a^{i-2j} a^{2j} b^j$

$S \rightarrow AB$

$A \rightarrow a | aA$

$B \rightarrow aaBb | \epsilon$

错误人数较少

错误原因在于忽略了 $i > 2j+1$

3. $\{\omega \mid \omega \in \{a,b,c\}^* \text{ and the numbers of } a\text{'s and } b\text{'s occurred in } \omega \text{ are odd.}\}$

根据 ab 个数的奇偶性，这里 0 代表偶数，1 代表奇数，*表示终态

	a	b
S	0	0
A	0	1
B	1	0
C*	1	1

$S \rightarrow aB \mid bA \mid cS$

$A \rightarrow aC \mid bS \mid cA$

$B \rightarrow aS \mid bC \mid cB$

$C \rightarrow aA \mid bB \mid \varepsilon \mid cC$

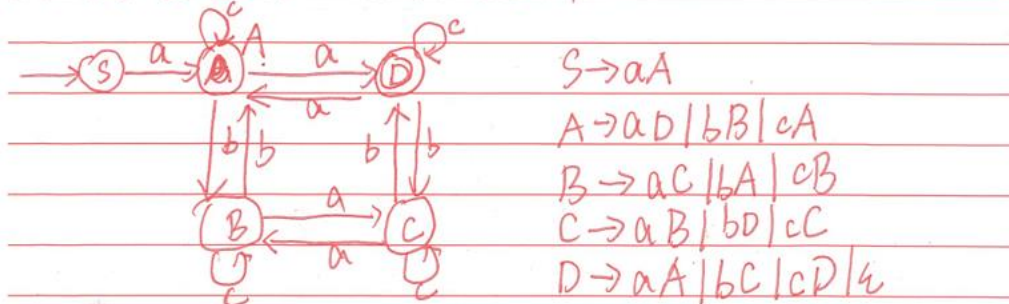
错误人数较少

错误原因在于不知道画电路图求解

错误原因在于忽略了终态

4. $\{\omega \mid \omega \in \{a,b,c\}^*, \omega \text{ is lead by } a \text{ and the numbers of } a\text{'s and } b\text{'s occurred in } \omega \text{ are even}\}$

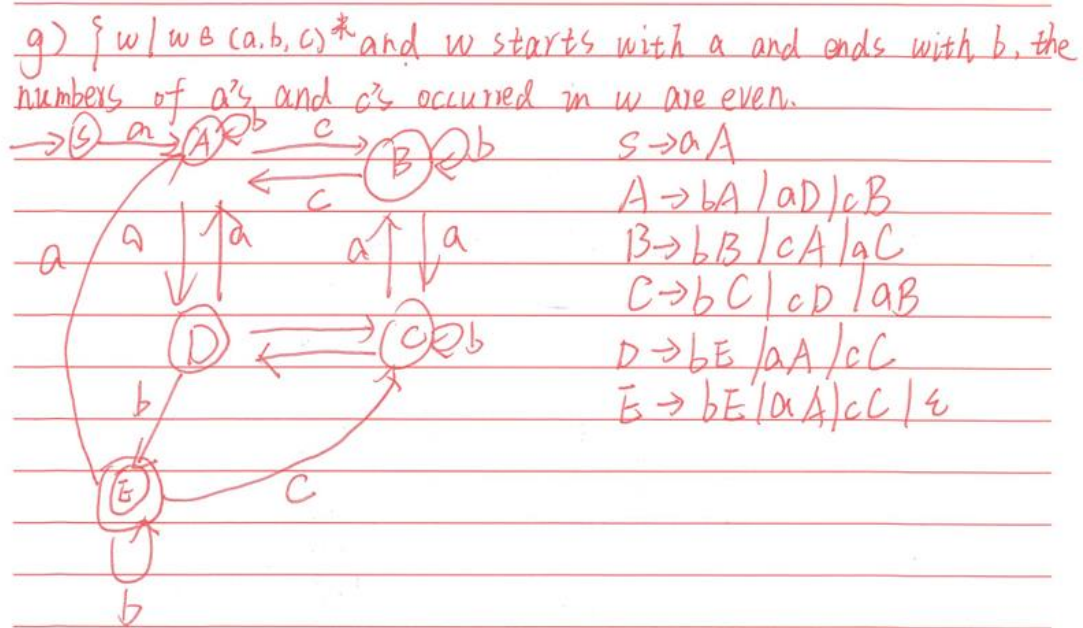
e) $\{w \mid w \in \{a,b,c\}^*, w \text{ is lead by } a \text{ and the numbers of } a\text{'s and } b\text{'s occurred in } w \text{ are even}\}$



错误人数较少

错误原因在于忽略了终态

5. $\{\omega \mid \omega \in \{a,b,c\}^*, \text{ and } \omega \text{ starts with } a \text{ and ends with } b, \text{ the numbers of } a\text{'s and } c\text{'s occurred in } \omega \text{ are even}\}$

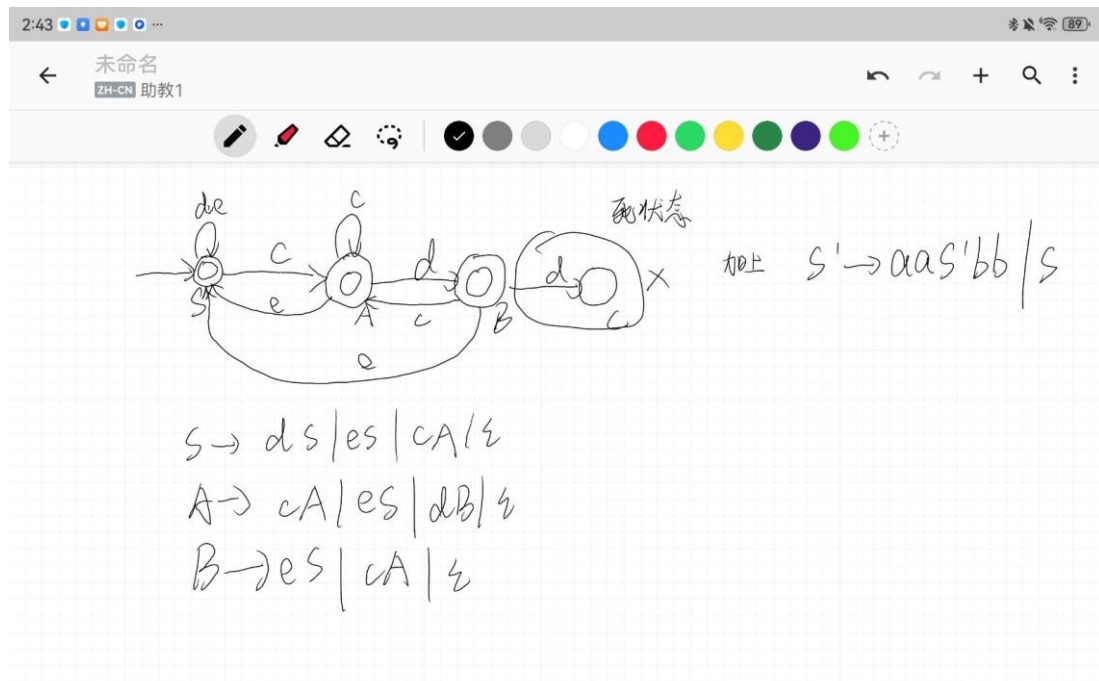


错误人数较少

错误原因在于不知道画电路图求解

错误原因在于忽略了终态

6. $\{a^{2m} \omega b^{2m} \mid m \geq 0, \omega \in \{c,d,e\}^* \text{ and } \omega \text{ does not contain the substring } cdd\}$.



错误人数较多

错误原因在于不知道怎么对问题状态进行划分或求解过于复杂

7. Construct a grammar for binary numbers whose corresponding decimal number can be divided by 3.

Handwritten solution for problem 7:

States and transitions:

- q_0 : 除以3余0的状态 (终止状态)
- q_1 : 除以3余1的状态
- q_2 : 除以3余2的状态
- 初始状态 q_3

Transitions:

- $q_3 \xrightarrow{0} q_0$
- $q_3 \xrightarrow{1} q_1$
- $q_0 \xrightarrow{0} q_0$
- $q_0 \xrightarrow{1} q_1$
- $q_1 \xrightarrow{0} q_2$
- $q_1 \xrightarrow{1} q_0$
- $q_2 \xrightarrow{0} q_1$
- $q_2 \xrightarrow{1} q_2$

Handwritten grammar rules:

- $q_3 \rightarrow 0 q_0 \mid 1 q_1$
- $q_0 \rightarrow 0 q_0 \mid 1 q_1$
- $q_1 \rightarrow 0 q_2 \mid 1 q_0$
- $q_2 \rightarrow 0 q_1 \mid 1 q_2$

错误人数较少

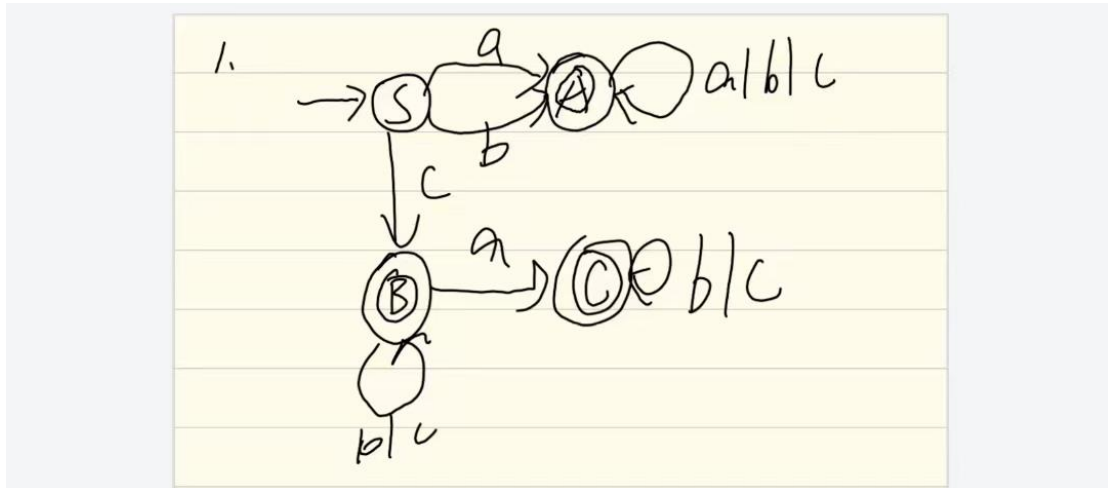
错误原因在于不知道余3分三个状态

8. For Boolean expressions in C language, please construct a regular grammar of their lexical patterns of at least 3 types of tokens, and a context-free grammar of sentential patterns containing at least 3 types of boolean operators.

错误人数较多

错误原因在于英文翻译不对，不知道要写两个文法

1. Construct a DFA M , which can accept the a, b, c strings which begin with a or b , or begin with c and contain at most one a . Please write a C++ function to implement the DFA.



错误同学较多

错误原因 1: 英文理解出错, 断句出现问题

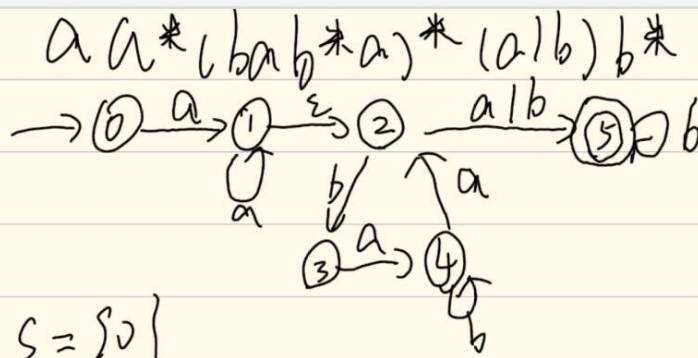
错误原因 2: 编写 C++ 代码未能体现状态转化, 不理解

DFA M 是什么

错误原因 3: NFA 转化为 DFA 及 DFA 最小化过程计算错误

3. Please construct a **DFA with minimum states** for the following regular expression.

(1) $aa^*(bab^*a)^*(a|b)b^*$



$S = \{0\}$

a b

$S \{0\}$ $A \{1,2\}$ $\emptyset$

$A \{1,2\}$ $B \{1,2,5\}$ $C \{3,5\}$

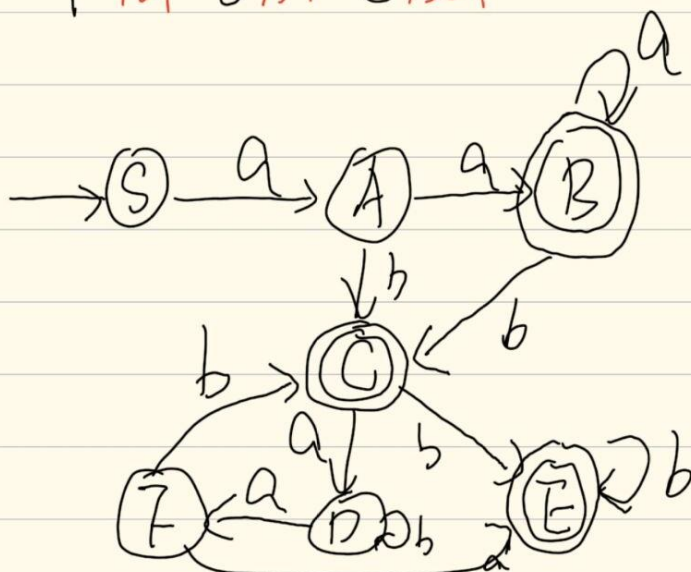
$*B \{1,2,5\}$ $B \{1,2,5\}$ $C \{3,5\}$

$*C \{3,5\}$ $D \{4\}$ $E \{5\}$

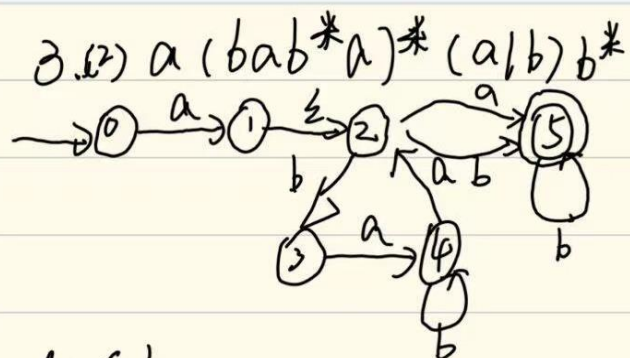
$D \{4\}$ $F \{2\}$ $D \{4\}$

$*E \{5\}$ $\emptyset$ $E \{5\}$

$F \{2\}$ $E \{5\}$ $C \{3,5\}$



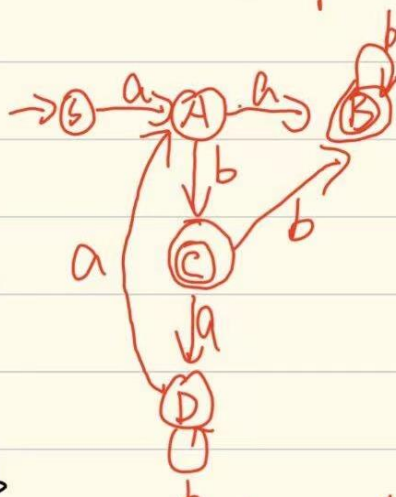
(2) $a(bab^*a)^*(a|b)b^*$



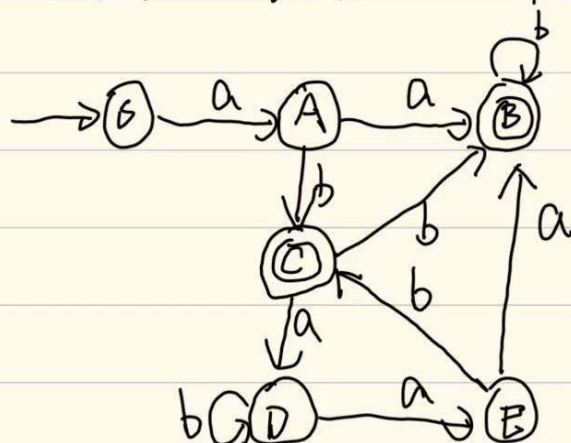
$S = \{0\}$

	a	b
$S \{0\}$	$A \{1,2\}$	$\emptyset$
$A \{1,2\}$	$B \{5\}$	$C \{3,4\}$
终 $B \{5\}$	$\emptyset$	$B \{5\}$
终 $C \{3,4\}$	$D \{4\}$	$B \{5\}$
$D \{4\}$	$E \{2\}$	$D \{4\}$
$E \{2\}$	$B \{5\}$	$C \{3,4\}$

A 为第 1 个状态



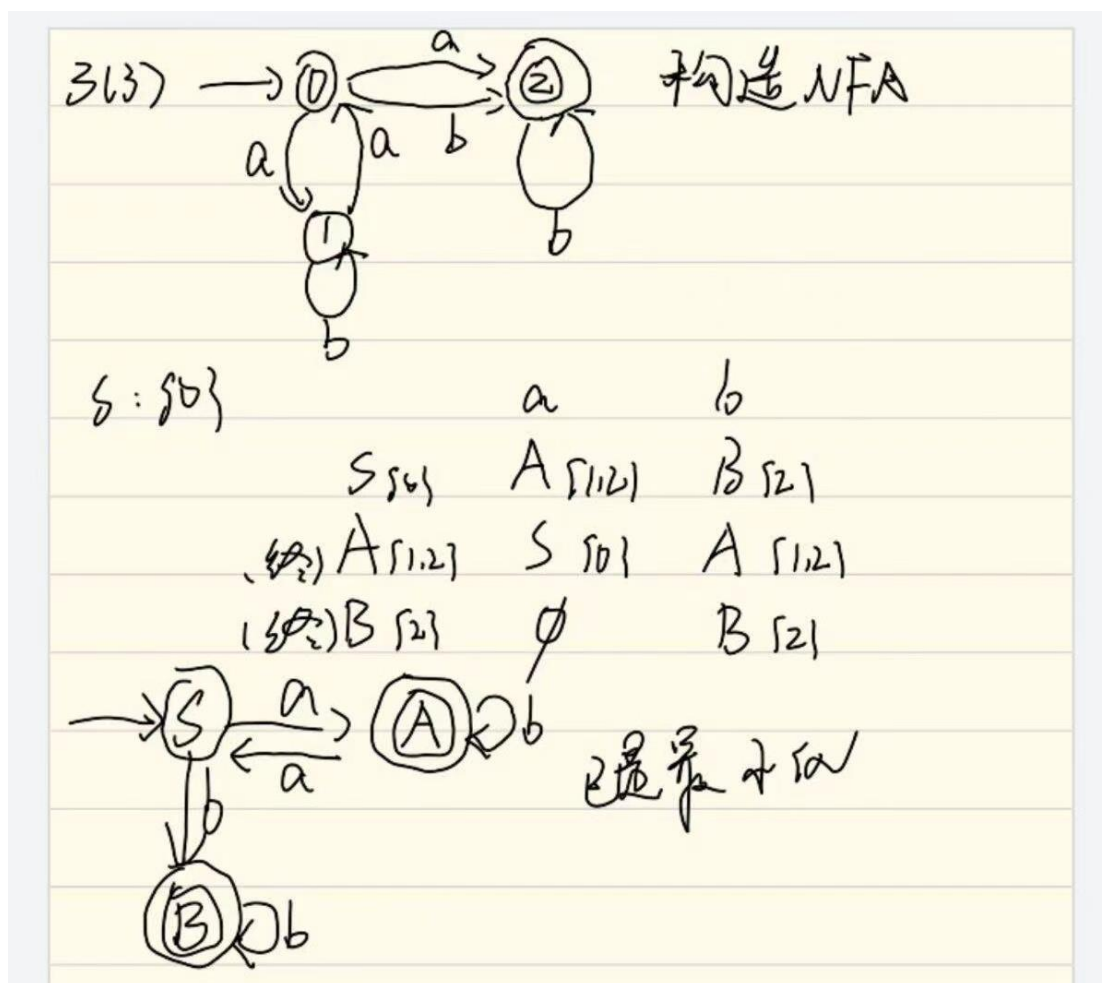
上图为最小化后



错误同学数量一般

错误原因主要是计算过程出错

(3) $(ab^*a)^*(a|b)b^*$



错误同学数量一般

错误原因主要是 DFA 最小化过程出错，对最小化过程理解有误

1. Eliminate the left recursion:

$$E \rightarrow ET+|T$$

$$T \rightarrow TF*|F$$

$$F \rightarrow E|i$$

Answer:

$$E \rightarrow ET+|T \quad T \rightarrow TF*|F \quad F \rightarrow E|i$$

(1) T, F, E

(2) $T \rightarrow TF*|F \Rightarrow T \rightarrow FT' \quad T' \rightarrow FT'|\epsilon$

$$F \rightarrow E|i$$

$$E \rightarrow ET+|FT' \Rightarrow E \rightarrow ET+|ET'|iT' \Rightarrow E \rightarrow iT'E' \quad E' \rightarrow T+E'|T'E'$$

(3) $E \rightarrow iT'E' \quad E' \rightarrow T+E'|T'E'|\epsilon \quad T \rightarrow FT' \quad T' \rightarrow FT'|\epsilon \quad F \rightarrow E|i$

错误人数很多

正确做法应该是先确定变量顺序，然后依次代入进行左递归的消除

同学们对于确定顺序之后，消除左递归该怎么做很模糊。不知道要代入已经确定的变量。或者有的人根本不知道要代入

2. Please analyse "ba(b)" by top-down method with backtracking, and tell us what are there weaknesses of top-down analysis, and why does LL(1) come? Then, judge whether the grammar is an LL(1) grammar by constructing the LL(1) parsing table.

$M \rightarrow MaH | H$

$H \rightarrow b(M)|(M)|b$

Answer:

(1) **Lift common left factor:** $M \rightarrow MaH | H$ $H \rightarrow bH' | (M)$ $H' \rightarrow (M) | \xi$

(2) **Elimit left recursion:** $M \rightarrow HM'$ $M' \rightarrow aHM' | \xi$

$H \rightarrow bH' | (M)$ $H' \rightarrow (M) | \xi$

(3) **First&Follow**

	FIRST	FOLLOW
M	{(, b}	{), #}
M'	{a, ε}	{), #}
H	{(, b}	{a,), #}
H'	{(, ε}	{a,), #}

(4) **LL(1) Parsing Table**

	a	(	)	b	#
M		$M \rightarrow HM'$		$M \rightarrow HM'$	
M'	$M' \rightarrow aHM'$		$M' \rightarrow \xi$		$M' \rightarrow \xi$
H		$H \rightarrow (M)$		$H \rightarrow bH'$	
H'	$H' \rightarrow \xi$	$H' \rightarrow (M)$	$H' \rightarrow \xi$		$H' \rightarrow \xi$

(5) **IS A LL(1) GRAMMAR**

3. Please eliminate the left recursion and extract maximum common left factors (if there are) from the following context free grammar, and then decide the resulted grammar is whether a LL(1) grammar by **constructing the related LL(1) parsing table.**

$S \rightarrow iCtS | iCtSeS | a$

$C \rightarrow C \text{ or } D | D$

$D \rightarrow D \text{ or } E | E$

$E \rightarrow (C) | b$

Answer:

$S \rightarrow iCtS | iCtSeS | a \quad C \rightarrow C \text{ or } D | D \quad D \rightarrow D \text{ or } E | E \quad E \rightarrow (C) | b$

(5) **Lift common left factor:** $S \rightarrow iCtSS' | a \quad S' \rightarrow eS | \epsilon \quad C \rightarrow C \text{ or } D | D \quad D \rightarrow D \text{ or } E | E$
 $E \rightarrow (C) | b$

(6) **Elimit left recursion:** $S \rightarrow iCtSS' | a \quad S' \rightarrow eS | \epsilon \quad C \rightarrow DC' \quad C' \rightarrow \text{or } DC' | \epsilon \quad D \rightarrow ED'$
 $D' \rightarrow \text{or } ED' | \epsilon \quad E \rightarrow (C) | b$

(7) **First&Follow**

	FIRST	FOLLOW
S	{i, a}	{e, #}
S'	{e, ϵ }	{e, #}
C	{(, b}	{t,)}
C'	{or, ϵ }	{t,)}
D	{(, b}	{or, t,)}
D'	{or, ϵ }	{or, t,)}
E	{(, b}	{or, t,)}

(8) LL(1) Parsing Table

	i	t	a	or	(	)	b	e	#
S	$S \rightarrow iCtS$ S'		$S \rightarrow a$						
S'								$S' \rightarrow eS \mid \epsilon$	$S' \rightarrow \epsilon$
C					$C \rightarrow DC$,		$C \rightarrow DC$,		
C'		$C' \rightarrow \epsilon$		$C' \rightarrow orDC'$		$C' \rightarrow \epsilon$			
D					$D \rightarrow ED$,		$D \rightarrow ED$,		
D'		$D' \rightarrow \epsilon$		$D' \rightarrow orED' \mid \epsilon$		$D' \rightarrow \epsilon$			
E					$E \rightarrow (C)$		$E \rightarrow b$		

(5) IS NOT A LL(1) GRAMMAR

4. Eliminate the left recursion:

$E \rightarrow ET+|T$

$T \rightarrow TF*|F$

$F \rightarrow E|i$

Answer:

$E \rightarrow ET+|T \quad T \rightarrow TF*|F \quad F \rightarrow E|i$

(4) T, F, E

(5) $T \rightarrow TF*|F \Rightarrow T \rightarrow FT' \quad T' \rightarrow FT'|\epsilon$

$F \rightarrow E|i$

$E \rightarrow ET+|FT' \Rightarrow E \rightarrow ET+|ET'|iT' \Rightarrow E \rightarrow iT'E' \quad E' \rightarrow T+E'|T'E'$

(6) $E \rightarrow iT'E' \quad E' \rightarrow T+E'|T'E'|\epsilon \quad T \rightarrow FT' \quad T' \rightarrow FT'|\epsilon \quad F \rightarrow E|i$

错误人数很多

正确做法应该是先确定变量顺序，然后依次代入进行左递归的消除

同学们对于确定顺序之后，消除左递归该怎么做很模糊。不知道要代入已经确定的变量。或者有的人根本不知道要代入

5. Please analyse “ba(b)” by top-down method with backtracking, and tell us what are there weaknesses of top-down analysis, and why does LL(1) come? Then, judge whether the grammar is an LL(1) grammar by constructing the LL(1) parsing table.

$M \rightarrow MaH | H$

$H \rightarrow b(M) | (M) | b$

Answer:

(9) **Lift common left factor:** $M \rightarrow MaH | H$ $H \rightarrow bH' | (M)$ $H' \rightarrow (M) | \xi$

(10) **Elimit left recursion:** $M \rightarrow HM'$ $M' \rightarrow aHM' | \xi$

$H \rightarrow bH' | (M)$ $H' \rightarrow (M) | \xi$

(11) **First&Follow**

	FIRST	FOLLOW
M	{(, b}	{),#}
M'	{a, ε}	{),#}
H	{ (, b}	{a,),#}
H'	{(, ε}	{a,),#}

(12) **LL(1) Parsing Table**

	a	(	)	b	#
M		$M \rightarrow HM'$		$M \rightarrow HM'$	
M'	$M' \rightarrow aHM'$		$M' \rightarrow \xi$		$M' \rightarrow \xi$
H		$H \rightarrow (M)$		$H \rightarrow bH'$	
H'	$H' \rightarrow \xi$	$H' \rightarrow (M)$	$H' \rightarrow \xi$		$H' \rightarrow \xi$

(5) **IS A LL(1) GRAMMAR**

6. Please eliminate the left recursion and extract maximum common left factors (if there are) from the following context free grammar, and then decide the resulted grammar is whether a LL(1) grammar by **constructing the related LL(1) parsing table**.

$S \rightarrow iCtS|iCtSeS|a$

$C \rightarrow C \text{ or } D|D$

$D \rightarrow D \text{ or } E|E$

$E \rightarrow (C) | b$

Answer:

$S \rightarrow iCtS|iCtSeS|a$ $C \rightarrow C \text{ or } D|D$ $D \rightarrow D \text{ or } E|E$ $E \rightarrow (C) | b$

(13)**Lift common left factor:** $S \rightarrow iCtSS'|a$ $S' \rightarrow eS|\epsilon$ $C \rightarrow C \text{ or } D|D$ $D \rightarrow D \text{ or } E|E$
 $E \rightarrow (C) | b$

(14)**Elimit left recursion:** $S \rightarrow iCtSS'|a$ $S' \rightarrow eS|\epsilon$ $C \rightarrow DC'$ $C' \rightarrow \text{or } DC'|\epsilon$ $D \rightarrow ED'$
 $D' \rightarrow \text{or } ED'|\epsilon$ $E \rightarrow (C) | b$

(15)**First&Follow**

	FIRST	FOLLOW
S	{i, a}	{e, #}
S'	{e, ϵ }	{e, #}
C	{(, b}	{t,)}
C'	{or, ϵ }	{t,)}
D	{(, b}	{or, t,)}
D'	{or, ϵ }	{or, t,)}
E	{(, b}	{or, t,)}

(16)**LL(1) Parsing Table**

	i	t	a	or	(	)	b	e	#
S	$S \rightarrow iCtS$ S'		$S \rightarrow a$						
S'								$S' \rightarrow eS $ ϵ	$S' \rightarrow$ ϵ
C					$C \rightarrow D$ C'		$C \rightarrow D$ C'		
C'		$C' \rightarrow$ ϵ		$C' \rightarrow \text{or } DC'$		$C' \rightarrow$ ϵ			
D					$D \rightarrow E$ D'		$D \rightarrow E$ D'		
D'		$D' \rightarrow$ ϵ		$D' \rightarrow \text{or } ED'$ $ \epsilon$		$D' \rightarrow$ ϵ			
E					$E \rightarrow (C)$		$E \rightarrow b$		

(5) **IS NOT A LL(1) GRAMMAR**

7. Construct a predictive table for the grammar:

$\text{bexpr} \rightarrow \text{bexpr or bterm} | \text{bterm}$

$\text{bterm} \rightarrow \text{bterm and bfactor} | \text{bfactor}$

$\text{bfactor} \rightarrow \text{not bfactor} | (\text{bexpr}) | \text{true} | \text{false}$

(1) Lift left factors

(2) Eliminate left recursion

$\text{bexpr} \rightarrow \text{bterm bexpr}'$

$\text{bexpr}' \rightarrow \text{or bterm bexpr}' | \epsilon$

$\text{bterm} \rightarrow \text{bfactor bterm}'$

$\text{bterm}' \rightarrow \text{and bfactor bterm}' | \epsilon$

$\text{bfactor} \rightarrow \text{not bfactor} | (\text{bexpr}) | \text{true} | \text{false}$

(3) FIRST&FOLLOW

	FIRST	FOLLOW
bexpr	{not, (, true, false}	{#,)}
bexpr'	{or, ε}	{#,)}
bterm	{not, (, true, false}	{or, #,)}
bterm'	{and, ε}	{or, #,)}
bfactor	{not, (, true, false}	{or, #,), and}

(4) Predictive Table

	not	true	false	or	(	)	and	#
bexpr	$\text{bexpr} \rightarrow \text{bterm bexpr}'$	$\text{bexpr} \rightarrow \text{bterm bexpr}'$	$\text{bexpr} \rightarrow \text{bterm bexpr}'$		$\text{bexpr} \rightarrow \text{bterm bexpr}'$			
bexpr'				$\text{bexpr}' \rightarrow \text{or bterm bexpr}'$		$\text{bexpr}' \rightarrow \epsilon$		$\text{bexpr}' \rightarrow \epsilon$
bterm	$\text{bterm} \rightarrow \text{bfactor bterm}'$	$\text{bterm} \rightarrow \text{bfactor bterm}'$	$\text{bterm} \rightarrow \text{bfactor bterm}'$		$\text{bterm} \rightarrow \text{bfactor bterm}'$			
bterm'				$\text{bterm}' \rightarrow \epsilon$		$\text{bterm}' \rightarrow \epsilon$	$\text{bterm}' \rightarrow \text{and bfactor bterm}'$	$\text{bterm}' \rightarrow \epsilon$
bfactor	$\text{bfactor} \rightarrow \text{not bfactor}$	$\text{bfactor} \rightarrow \text{true}$	$\text{bfactor} \rightarrow \text{false}$		$\text{bfactor} \rightarrow (\text{bexpr})$			

错误人数不多

主要错误原因是不进行左递归的消除就直接进行 First 集合和 Follow 集合的构造了，提取左公因子和消除左递归应该是必要过程吧？

8. Please **construct a LR(1) parsing table for the following ambiguous grammar with the additional conditions** that **else** dangles with the closest previous unmatched **if** , ; has the property of left associative.

$S \rightarrow \text{if } S \text{ else } S \mid \text{if } S \mid S; S \mid a$

R0: $S' \rightarrow S$

R1: $S \rightarrow \text{if } S \text{ else } S$

R2: $S \rightarrow \text{if } S$

R3: $S \rightarrow S; S$

R4: $S \rightarrow a$

	Action					GOTO
	if	else	;	a	#	S
0	S2			S3		1
1			S4		ACC	
2	S6			S7		5
3			R4		R4	
4	S2			S3		8
5		S9	R2/S10		R2	
6	S6			S7		11
7		R4	R4		R4	
8			R3/S4		R3	
9	S2			S3		12
10	S6			S7		13
11		R2/S14	R2/S10		R2	
12			S4/R1		R1	
13		R3	R3/S10		R3	
14	S6			S7		15
15		R1	R1/S10		R1	

根据条件 1 当存在移进 else 和 R2 规约冲突时，优先移进 else

根据条件 2, ; 具有左结合性可知，当移进; 和 R3 归约存在冲突时优先 R3 规约但是移进; 和 R1 规约、R2 规约任然存在冲突

	Action					GOTO
	if	else	;	a	#	S
0	S2			S3		1
1			S4		ACC	
2	S6			S7		5
3			R4		R4	
4	S2			S3		8
5		S9	R2/S10		R2	
6	S6			S7		11
7		R4	R4		R4	
8			R3		R3	
9	S2			S3		12
10	S6			S7		13
11		S14	R2/S10		R2	
12			R1/S4		R1	
13		R3	R3		R3	
14	S6			S7		15

15		R1	R1/S10		R1	
----	--	----	--------	--	----	--

错误人数非常恐怖

同学们对于条件理解很不到位，错误地将：具有左结合性认为是一遇到；就进行归约。用仅仅两个条件就把 LR(1)预测表的冲突全部消除了

9. Please construct a LR(1) parsing table for the following ambiguous grammar with the additional conditions that **or** has higher precedence than **do**, **;** has higher precedence than **do**, **;** has higher precedence than **or**, **;** has the property of left associative.

$S \rightarrow \text{do } S \text{ or } S | \text{do } S | S ; S | a$

R0: $S' \rightarrow S$

R1: $S \rightarrow \text{do } S \text{ or } S$

R2: $S \rightarrow \text{do } S$

R3: $S \rightarrow S ; S$

R4: $S \rightarrow a$

构造 LR1 分析表

	Action					GOTO
	do	or	;	a	#	S
0	S2			S3		1
1			S4		ACC	
2	S6			S7		5
3			R4		R4	
4	S2			S3		8
5		S9	R2/S10		R2	
6	S6			S7		11
7		R4	R4		R4	
8			R3/S4		R3	
9	S2			S3		12
10	S6			S7		13
11		R2/S14	R2/S10		R2	
12			S4/R1		R1	
13		R3	R3/S10		R3	
14	S6			S7		15
15		R1	R1/S10		R1	

因为优先级为: $>\text{or}>\text{do}$,

所以 R3 规约优先级高于移进 do、or

移进; 优先级高于 R1、R2 规约

移进 or 优先级高于 R2 规约

; 具有左结合性可知, 当移进; 和 R3 归约存在冲突时优先 R3 规约

	Action					GOTO
	do	or	;	a	#	S
0	S2			S3		1
1			S4		ACC	
2	S6			S7		5
3			R4		R4	
4	S2			S3		8
5		S9	S10		R2	
6	S6			S7		11
7		R4	R4		R4	
8			R3		R3	
9	S2			S3		12
10	S6			S7		13

11		S14	S10		R2	
12			S4		R1	
13		R3	R3		R3	
14	S6			S7		15
15		R1	S10		R1	

本题错误人数较多，主要是 LR(1)预测表计算错误和根据题目给出的条件判断消除移进-归约冲突的优先级判断错误

一、Please construct **an annotated parse tree** for the input string $4*5+6$ where the syntax-directed definition is as following

Productions

$E \rightarrow E1 * T$

$E \rightarrow T$

$T \rightarrow T1 + F$

$T \rightarrow F$

$F \rightarrow i$

Semantic Rules

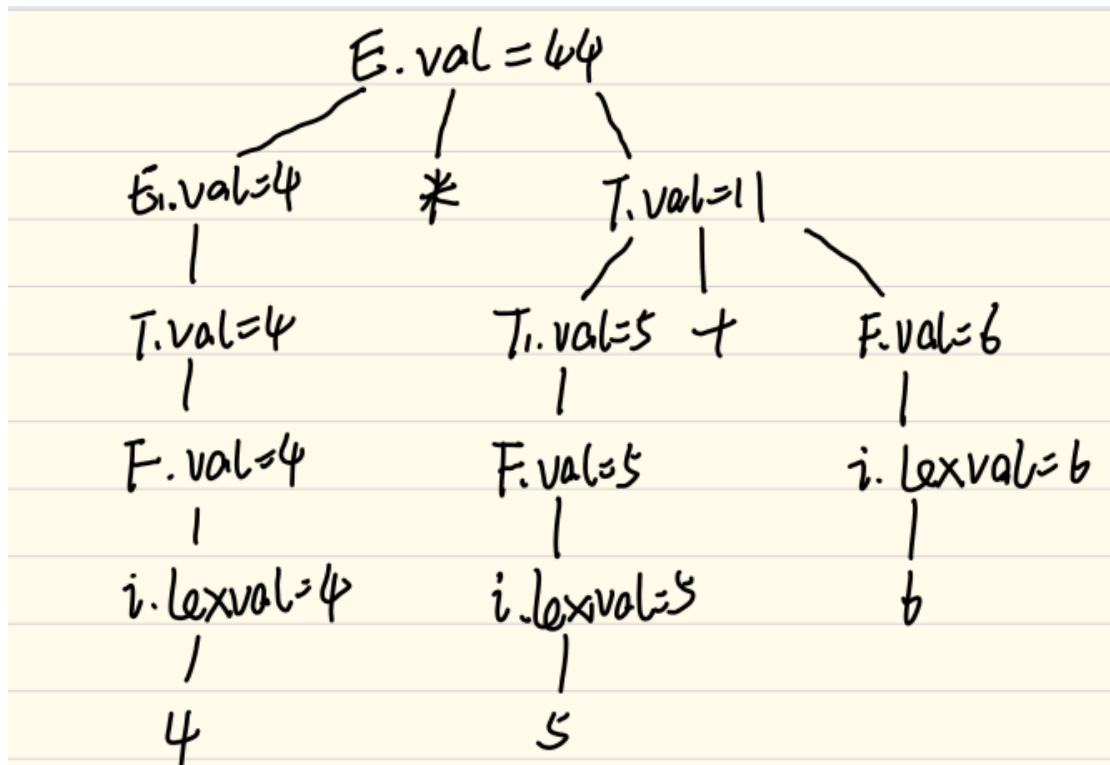
$E.val = E1.val * T.val$

$E.val = T.val$

$T.val = T1.val + F.val$

$T.val = F.val$

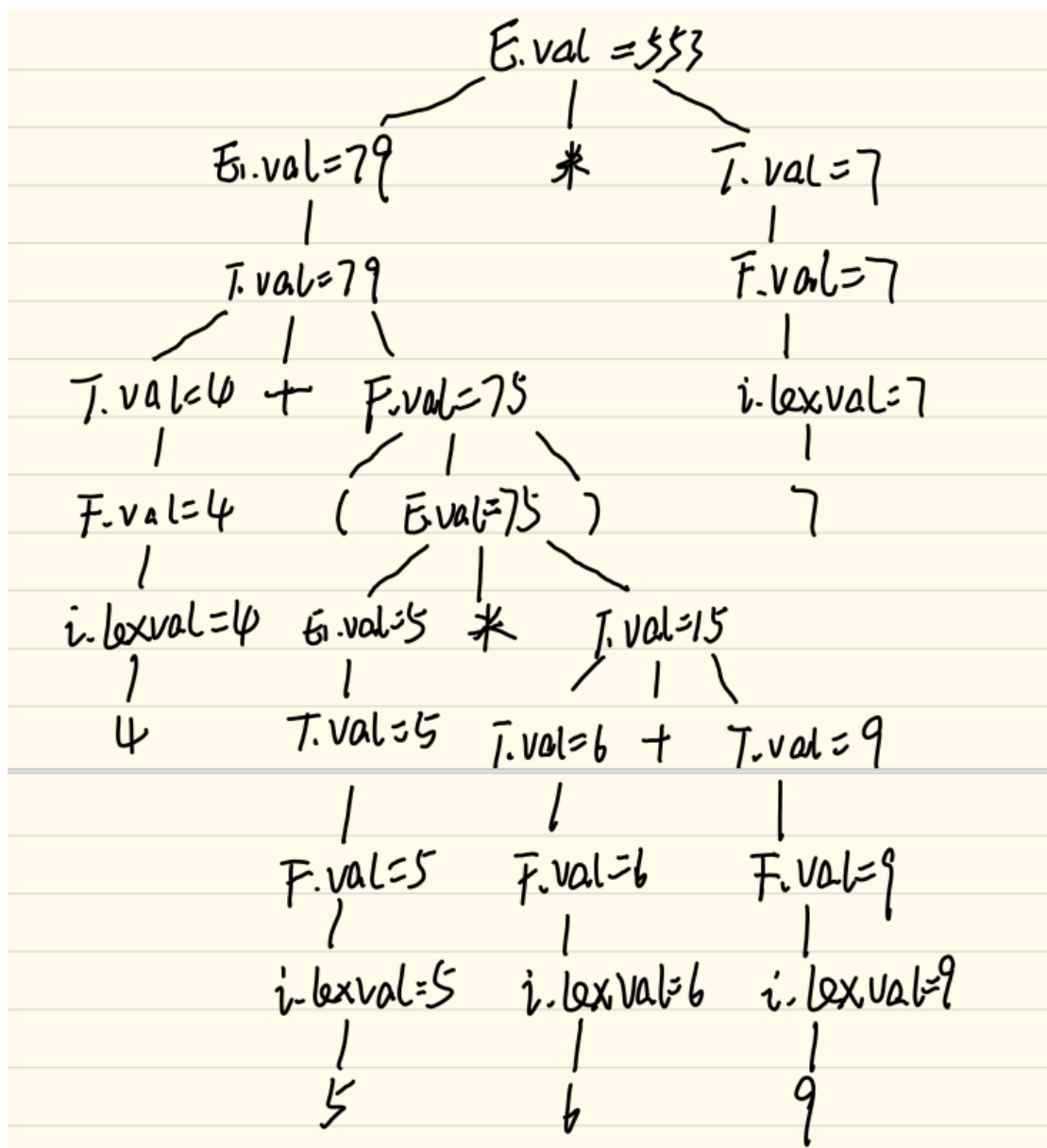
$F.val = i.lexval$



第一题和第二题都比较简单，基本没什么错误

二、Please construct **an annotated parse tree** for the input string $4+(5*6+9)*7$ where the syntax-directed definition is as following

Productions	Semantic Rules
$E \rightarrow E1 * T$	$E.val = E1.val * T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T1 + F$	$T.val = T1.val + F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow i$	$F.val = i.lexval$



5. Please translate the following program fragment into three-address code using the form of short circuit code.

```
i=2;
loop=0;
while (loop==0 && i<=10) {
    j=1;
    while (loop ==0 && j<i)
        if (a[i,j] == x)
            loop=1;
        else j=j+1;
    if (loop==0) i=i+1;
}
```

Notes: Here we assume that the declaration of array a is array [0..10,0..10], each data element of array a would use 2 storage units, and the start address of array A's storage area is a.

参考答案:

```
(1)i:=2
(2)loop:=0
(3)if loop!=0 goto 22
(4)if i>10 goto 22
(5)j:=1
(6)if loop !=0 goto 18
(7)if j ≥ i goto 18
    求出 a[i, j]
(8)t1:=addrA-22
(9)t2:=i*22
(10)t3:=j*2
(11)t4:=t2+t3
(12)t5:=t1[t4]
(13)if t5!=x goto 16
(14)loop:=1
(15)goto 18
(16)t6:=j+1
(17)j:=t6
(18)if loop!=0 goto 21
(19)t7:=i+1
(20)i:=t7
(21)goto 3
(22)
```

第三题和第四题题目类型比较相似，错误类型也一致

错误原因：类似 i++，j++ 没有使用中间变量以及转移 goto 弄错

6. Translate the following program fragment into three-address code of quadruple representation, using short circuit code and back-patching techniques. Please keep the traces of merging and backpatching.

```
i=2;
m=0;
loop=0;
while (loop==0 && i<=10) {
    j=1;
    while (loop ==0 && j<=i)
        if (a[i,j] != a[j,i]) //"!=" means "not equal to"
        {
            loop=1;
            m=1;
        }
    else j=j+1;
    if (loop==0) i=i+1;
}
```

```
    }
    else j=j+1;
    if (loop==0) i=i+1;
}
```

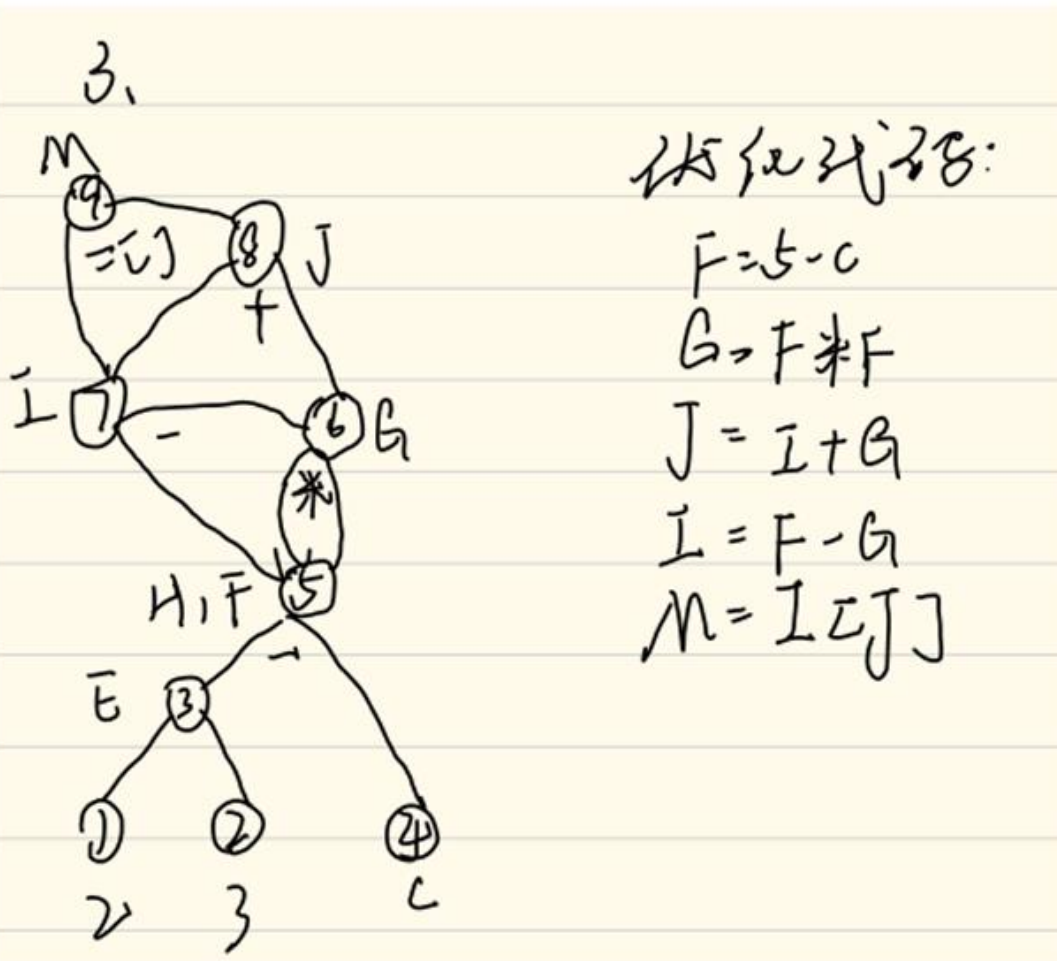
Notes: Here we assume that the declaration of array A is array [1..10,1..10], each data element of array a would **only use 1 storage unit**, and the start address of array a's storage area is a.

参考答案:

- (1) i:=2
- (2) m:=0
- (3) loop:=0
- (4) if loop!=0 *goto* 28
- (5) if i>10 *goto* 28
- (6) j:=1
- (7) if loop!=0 *goto* 24
- (8) if j>i *goto* 24
- //计算 a[i,j]和 a[j,i]
- (9) t1:=addrA-11
- (10) t2:=i*10
- (11) t3:=j

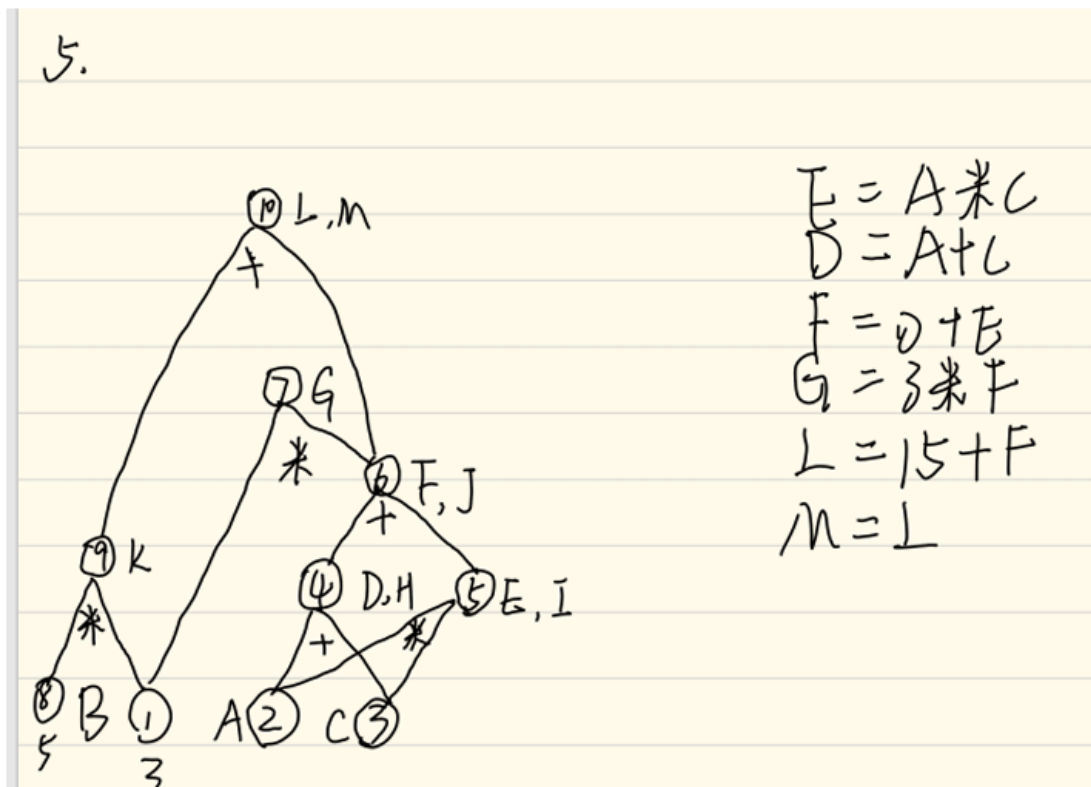
(12) $t4:=t2+t3$
(13) $t5:=t1[t4]$
*(14) $t6:=j*10$*
(15) $t7:=i$
(16) $t8:=t6+t7$
(17) $t9:=t1[t9]$
(18) if $t5==t9$ goto 22
(19) loop:=1
(20) m:=1
(21) goto 24
(22) $t10:=j+1$
(23) $j:=t10$
(24) if loop!=0 goto 27
(25) $t11:=i+1$
(26) $i:=t11$
(27) goto 4

1. Please construct the DAG for the following basic block, and we assume that only variable “M” is live on exit, please optimize the block and rewrite the block in optimized code form

$$E=2+3$$
$$F = E - C$$
$$G = F * F$$
$$H=E-C$$
$$I = H - G$$
$$J=I+G$$
$$M=I[J]$$


2. Please construct the DAG for the following basic block, optimize the block and rewrite the block in optimized code form. Note that we assume only Variable G,L,M would be used later

$B=3$
 $D=A+C$
 $E=A*C$
 $F=D+E$
 $G=B*F$
 $H=A+C$
 $I=A*C$
 $J=H+I$
 $K=B*5$
 $L=K+J$
 $M=L$



3. Please construct the DAG for the following basic block, and we assume that only variable “M” would be used later, please optimize the block and rewrite the block in optimized code form.

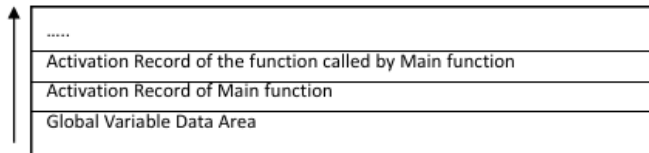
```
T1=1*20
T2=T1+1
T3=T2*4

T4=addrA-T3
T5=i*20
T6=T5+j
T7=T6*2
T8=1*20
T9=T8+1
T10=T9*4
T11=addrB-T10
T12=i*20
T13=T12+j
T14=T13*2
T15=T11[T14]
A=3
T16=T15*A
T4[T7]=T16
M=T4[T7]
M=M*M
L=M
```

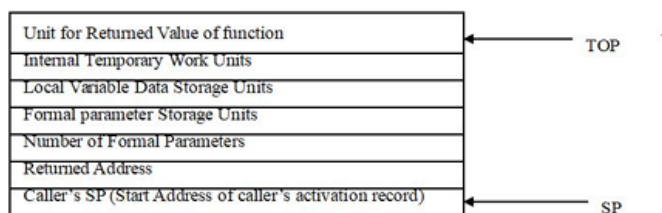
```
T11=addrB-84
T5=i*20
T6=T5+j
T7=2*T6
T15=T11[T7]
T16=3*T15
M=T16*T16
```

3. The storage organization and the form of activation record used in C language are as following. Please construct the run-time stack map when it gets the maximum size for the first time for the following C program.

Storage Organization of C Language



The Form of Activation Record of any a function in C



The C program is as the following:

```
#include <stdio.h>
```

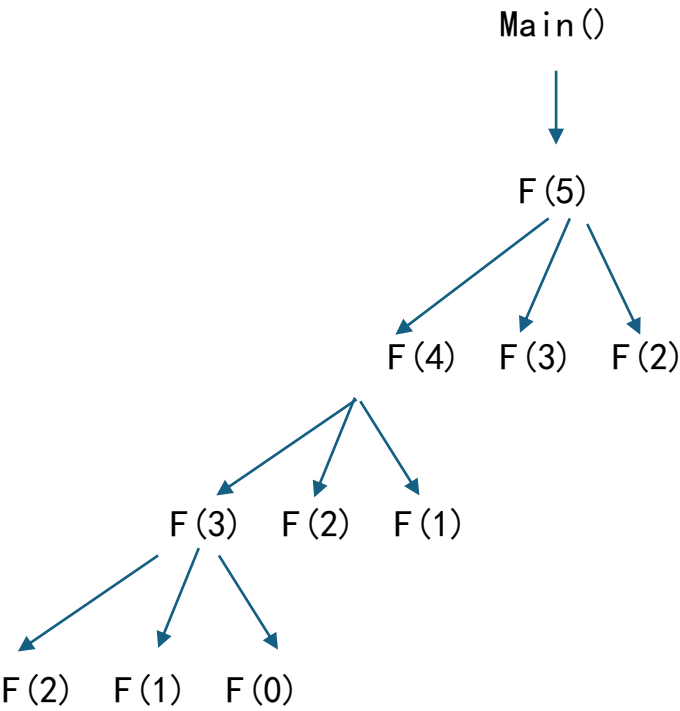
```
int x,y;
int main()
{ x=5; y=f(x);}
int f(int n)
{ if (n<=1)
    return 1;
  else
    if ( n==2) return 2;
  else
    { int t1,t2,t3,t4,t;
      t1=f(n-1); t2=f(n-2); t3=f(n-3); t4=t1+t2;t=t3+t4; return t
    }
}
```

Notes: 1) Here we assume that the caller's sp of Main function is the start address of global variable data area, and the returned address in the activation record of a function (including Main function) is filled by the operating system automatically, you might not care it.

2) The initial value of variable X is 5, the start address of stack used in the program is K.

3) The stack map may get its maximum size for several times, here we ask you draw the stack map at maximum size for the first time.

参考答案：



K+1	y	
K	x	5

Global variable

K+5	0
K+4	0
K+3	//
K+2	K

Main()

K+15	0	
K+14	t	
K+13	T4	
K+12	T3	
K+11	T2	
K+10	T1	
K+9	n	5
K+8	1	
K+7	//	
K+6	K+2	

F (5)

K+25	0	
K+24	t	
K+23	T4	
K+22	T3	
K+21	T2	
K+20	T1	
K+19	n	4
K+18	1	
K+17	//	
K+16	K+6	

F (4)

K+35	0	
K+34	t	
K+33	T4	
K+32	T3	
K+31	T2	
K+30	T1	
K+29	n	2
K+28	1	
K+27	//	
K+26	K+16	

F (3)

K+40	2	
K+39	n	2
K+38	1	
K+37	//	
K+36	K+26	

4. Please construct the run-time stack map (detailed map) when it gets the maximum size for the second time for the following C program

```
#include <stdio.h>

int x,y,z;

int main()
{
    x=5;
    y=8;
    z=f(x,y);
}

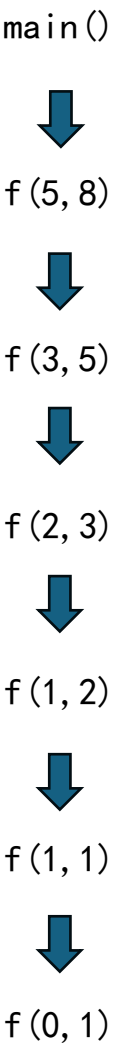
int f(int m, int n)
{
    if (m>=0 && n>=0)
    {
        if (m==0) return n;
        else if (n==0) return m;
```

```
        else
        {
            int t1,t2,t;
            if (m>=n) {
                t1=m-n;
                t2=n
            }
            else {
                t1=n-m;
                t2=m
            }
            t=f(t1,t2);
            return t
        }
    }
}
```

Notes: 1) Here we assume that the caller's sp of Main function is the start address of global variable data area, and the returned address in the activation record of a function (including Main function) is filled by the operating system automatically, you might not care it.

2) The start address of stack used in the program is K.

第一步：构建 activity tree:



第二步：构建 activity stack:

f(0, 1)
f(1, 1)
f(1, 2)
f(2, 3)
f(3, 5)
f(5, 8)
Main()
global variable

第三步：记录

K+2	z	
K+1	y	8
K	x	5

Global variable

K+6	0
K+5	0
K+4	//
K+3	K

Main()

K+15	0	
K+14	t	
K+13	T2	5
K+12	T1	3
K+11	n	8
K+10	M	5
K+9	2	
K+8	//	
K+7	K+3	

F (5, 8)

K+33	0	
K+32	t	
K+31	T2	2
K+30	T1	1
K+29	n	3
K+28	M	2
K+27	2	
K+26	//	
K+25	K+16	

F (2, 3)

K+24	0	
K+23	t	
K+22	T2	3
K+21	T1	2
K+20	n	5
K+19	M	3
K+18	2	
K+17	//	
K+16	K+7	

F (3, 5)

K+42	0	
K+41	t	
K+40	T2	1
K+39	T1	1
K+38	n	2
K+37	M	1
K+36	2	
K+35	//	
K+34	K+25	

F (1, 2)

K+51	0	
K+50	t	
K+49	T2	1
K+48	T1	0
K+47	n	1
K+46	M	1
K+45	2	
K+44	//	
K+43	K+34	

F (1, 1)

K+57	0	
K+56	n	1
K+55	M	0
K+54	2	
K+53	//	
K+52	K+43	

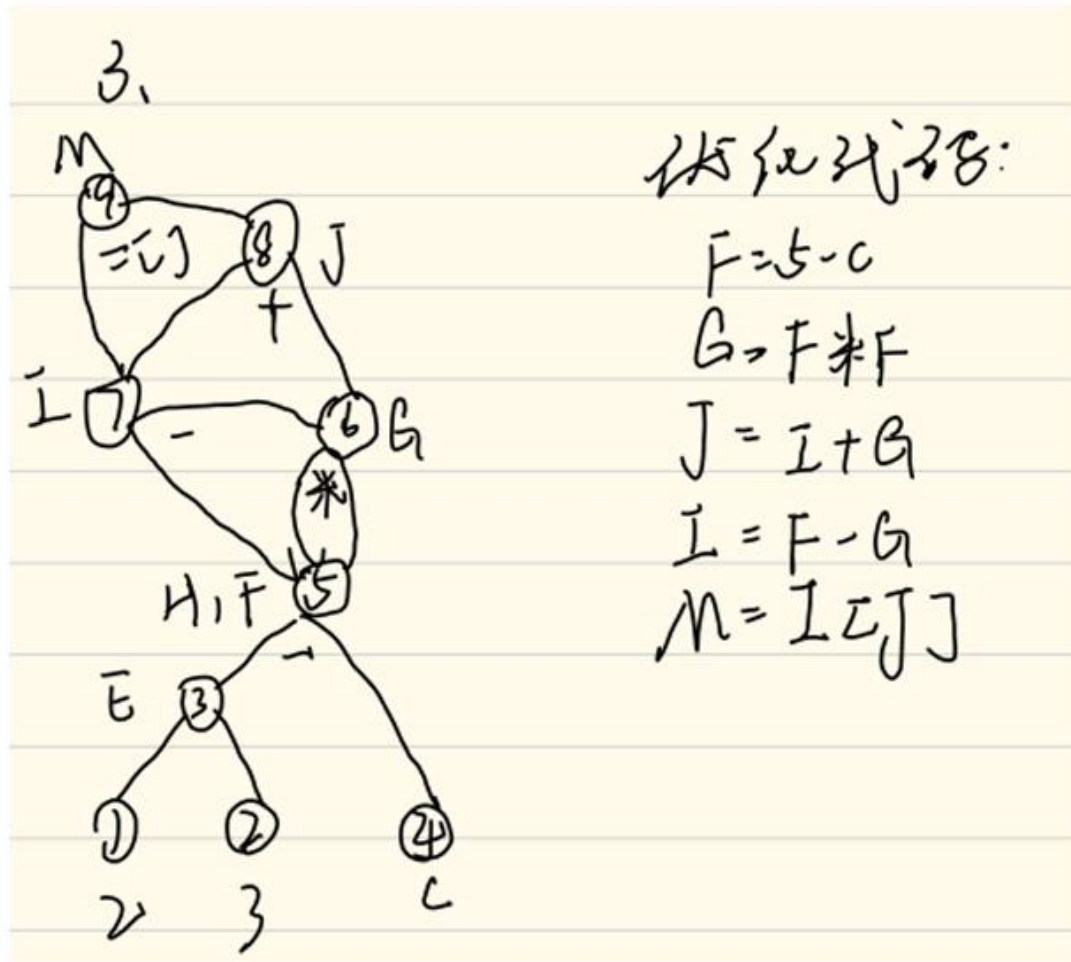
F (0, 1)

1. Please construct the DAG for the following basic block, and we assume that only variable “M” is live on exit, please optimize the block and rewrite the block in optimized code form

```

E=2+3
F=E-C
G=F*F
H=E-C
I=H-G
J=I+G
M=I[J]

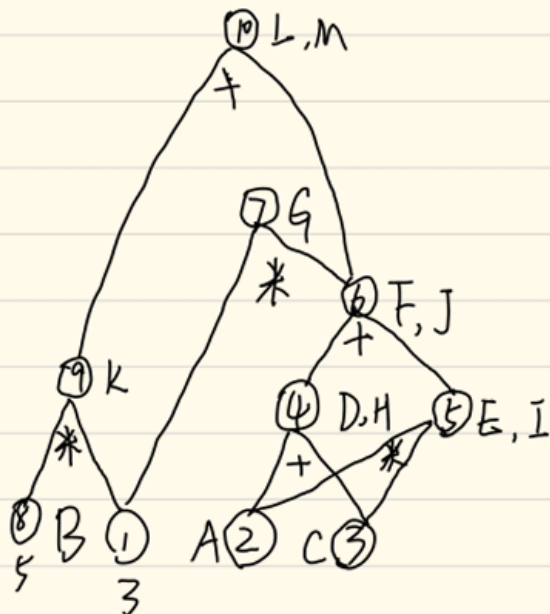
```



2. Please construct the DAG for the following basic block, optimize the block and rewrite the block in optimized code form. Note that we assume only Variable G,L,M would be used later

B=3
 D=A+C
 E=A*C
 F=D+E
 G=B*F
 H=A+C
 I=A*C
 J=H+I
 K=B*5
 L=K+J
 M=L

5.



$E = A * C$
 $D = A + C$
 $F = D + E$
 $G = 3 * F$
 $L = 15 + F$
 $M = L$

3. Please construct the DAG for the following basic block, and we assume that only variable "M" would be used later, please optimize the block and rewrite the block in optimized code form.

$T1 = 1 * 20$

$T2 = T1 + 1$

$T3 = T2 * 4$

~~T4=addrA-T3~~

T5=i*20

T6=T5+j

T7=T6*2

T8=1*20

T9=T8+1

T10=T9*4

T11=addrB-T10

T12=i*20

T13=T12+j

T14=T13*2

T15=T11[T14]

A=3

T16=T15*A

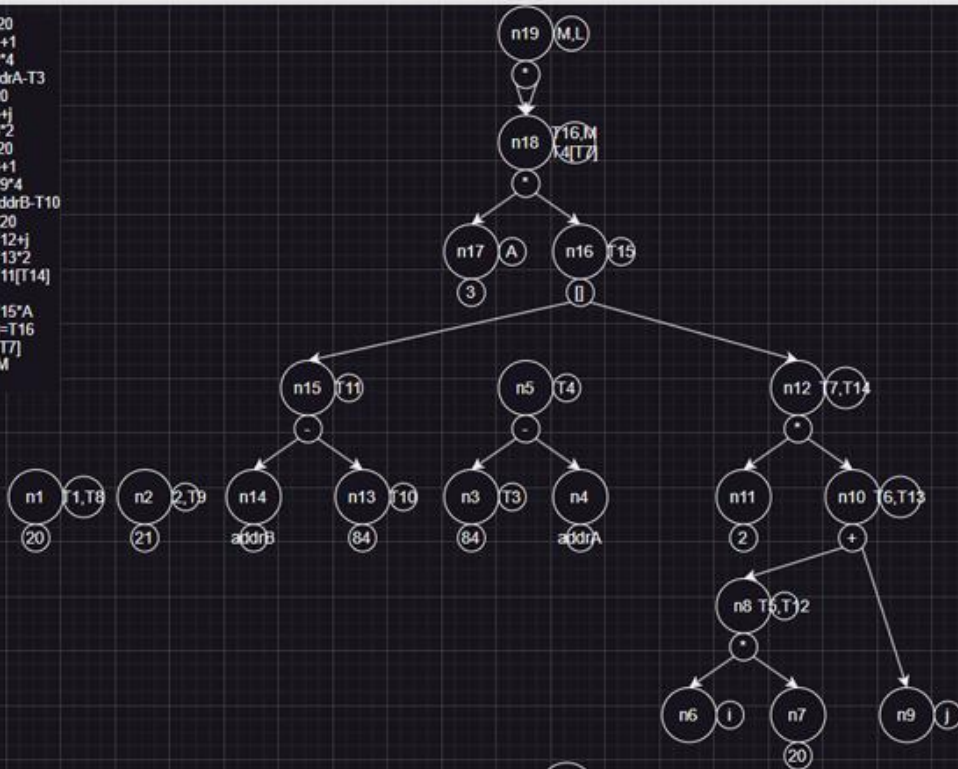
T4[T7]=T16

M=T4[T7]

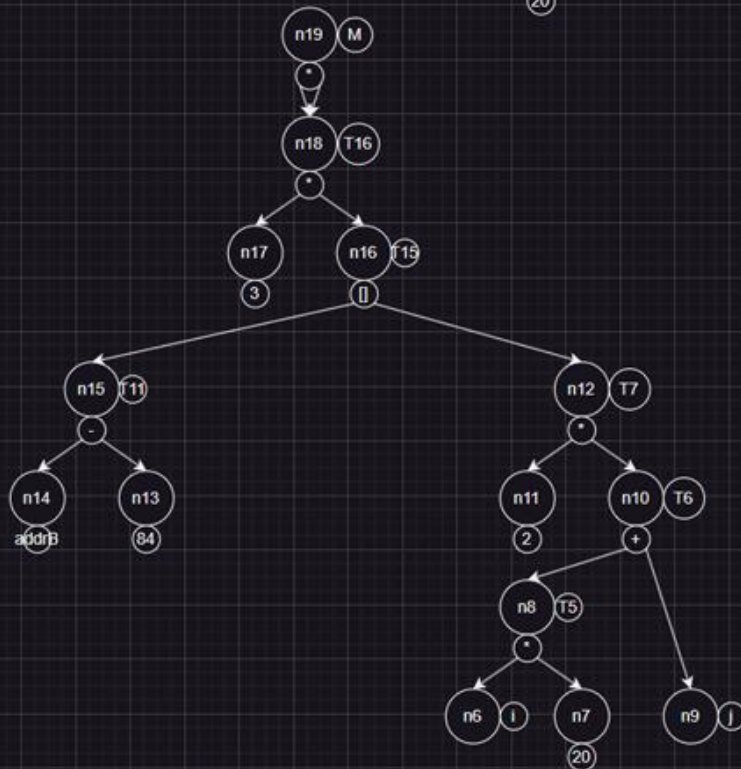
M=M*M

L=M

$T1 = i * 20$
 $T2 = T1 + 1$
 $T3 = T2 * 4$
 $T4 = \text{addrA} - T3$
 $T5 = i * 20$
 $T6 = T5 + j$
 $T7 = T6 * 2$
 $T8 = T7 * 20$
 $T9 = T8 + 1$
 $T10 = T9 * 4$
 $T11 = \text{addrB} - T10$
 $T12 = i * 20$
 $T13 = T12 + j$
 $T14 = T13 * 2$
 $T15 = T11[T14]$
 $A = 3$
 $T16 = T15 * A$
 $T4[T7] = T16$
 $M = T4[T7]$
 $M = M * M$
 $L = M$



$T11 = \text{addrB} - 84$
 $T5 = i * 20$
 $T6 = T5 + j$
 $T7 = 2 * T6$
 $T15 = T11[T7]$
 $T16 = 3 * T15$
 $M = T16 * T16$



1. Please explain the following terms in your own language

(1) Program and Process

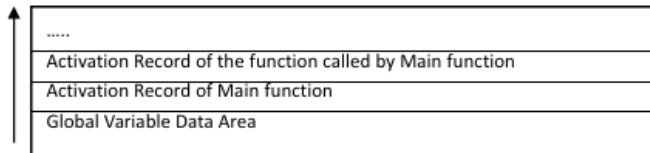
(2) Activation tree and Control stack

(3) Environment and State

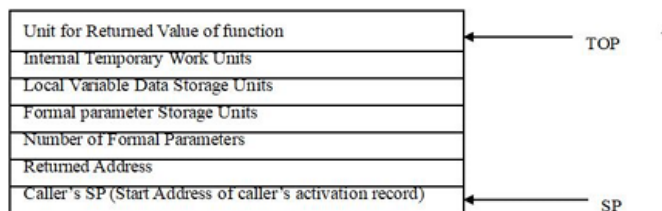
2. Please give the general map of the data area of a process, the general map of a function's record. A detailed introduction of the maps is needed.

3. The storage organization and the form of activation record used in C language are as following. Please construct the run-time stack map when it gets the maximum size for the first time for the following C program.

Storage Organization of C Language



The Form of Activation Record of any a function in C



The C program is as the following:

```
#include <stdio.h>
```

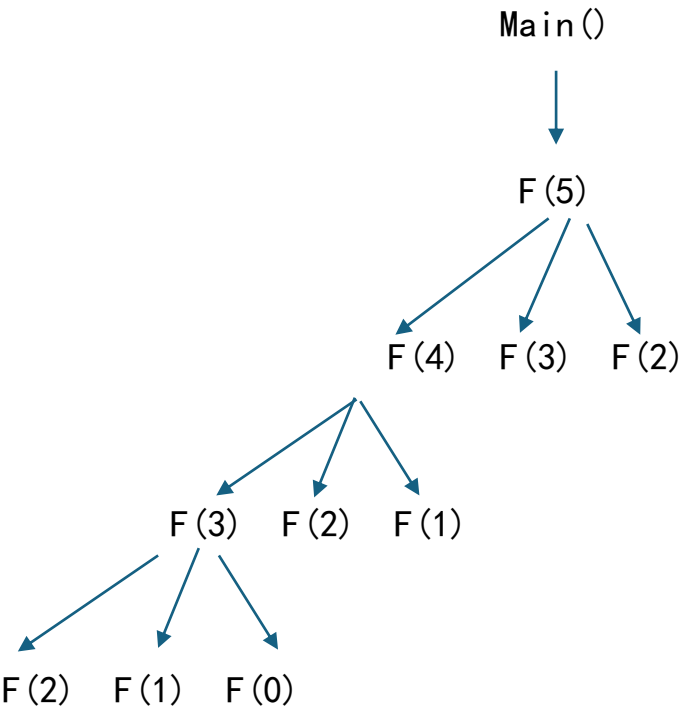
```
int x,y;
int main()
{ x=5; y=f(x);}
int f(int n)
{ if (n<=1)
    return 1;
  else
    if ( n==2) return 2;
  else
    { int t1,t2,t3,t4,t;
      t1=f(n-1); t2=f(n-2); t3=f(n-3); t4=t1+t2;t=t3+t4; return t
    }
}
```

Notes: 1) Here we assume that the caller's sp of Main function is the start address of global variable data area, and the returned address in the activation record of a function (including Main function) is filled by the operating system automatically, you might not care it.

2) The initial value of variable X is 5, the start address of stack used in the program is K.

3) The stack map may get its maximum size for several times, here we ask you draw the stack map at maximum size for the first time.

参考答案：



K+1	y	
K	x	5

Global variable

K+5	0
K+4	0
K+3	//
K+2	K

Main()

K+15	0	
K+14	t	
K+13	T4	
K+12	T3	
K+11	T2	
K+10	T1	
K+9	n	5
K+8	1	
K+7	//	
K+6	K+2	

F (5)

K+25	0	
K+24	t	
K+23	T4	
K+22	T3	
K+21	T2	
K+20	T1	
K+19	n	4
K+18	1	
K+17	//	
K+16	K+6	

F (4)

K+35	0	
K+34	t	
K+33	T4	
K+32	T3	
K+31	T2	
K+30	T1	
K+29	n	2
K+28	1	
K+27	//	
K+26	K+16	

F (3)

K+40	2	
K+39	n	2
K+38	1	
K+37	//	
K+36	K+26	

4. Please construct the run-time stack map (detailed map) when it gets the maximum size for the second time for the following C program

```
#include <stdio.h>

int x,y,z;

int main()
{
    x=5;
    y=8;
    z=f(x,y);
}

int f(int m, int n)
{
    if (m>=0 && n>=0)
    {
        if (m==0) return n;
        else if (n==0) return m;
```

```
        else
        {
            int t1,t2,t;
            if (m>=n) {
                t1=m-n;
                t2=n
            }
            else {
                t1=n-m;
                t2=m
            }
            t=f(t1,t2);
            return t
        }
    }
}
```

Notes: 1) Here we assume that the caller's sp of Main function is the start address of global variable data area, and the returned address in the activation record of a function (including Main function) is filled by the operating system automatically, you might not care it.

2) The start address of stack used in the program is K.

第一步：构建 activity tree:

main()



f(5, 8)



f(3, 5)



f(2, 3)



f(1, 2)



f(1, 1)



f(0, 1)

第二步：构建 activity stack:

f(0, 1)
f(1, 1)
f(1, 2)
f(2, 3)
f(3, 5)
f(5, 8)
Main()
global variable

第三步：记录

K+2	z	
K+1	y	8
K	x	5

Global variable

K+6	0
K+5	0
K+4	//
K+3	K

Main()

K+15	0	
K+14	t	
K+13	T2	5
K+12	T1	3
K+11	n	8
K+10	M	5
K+9	2	
K+8	//	
K+7	K+3	

F (5, 8)

K+33	0	
K+32	t	
K+31	T2	2
K+30	T1	1
K+29	n	3
K+28	M	2
K+27	2	
K+26	//	
K+25	K+16	

F (2, 3)

K+24	0	
K+23	t	
K+22	T2	3
K+21	T1	2
K+20	n	5
K+19	M	3
K+18	2	
K+17	//	
K+16	K+7	

F (3, 5)

K+42	0	
K+41	t	
K+40	T2	1
K+39	T1	1
K+38	n	2
K+37	M	1
K+36	2	
K+35	//	
K+34	K+25	

F (1, 2)

K+51	0	
K+50	t	
K+49	T2	1
K+48	T1	0
K+47	n	1
K+46	M	1
K+45	2	
K+44	//	
K+43	K+34	

F (1, 1)

K+57	0	
K+56	n	1
K+55	M	0
K+54	2	
K+53	//	
K+52	K+43	

F (0, 1)